*ptr ** ptr *** ptr

Runtime Polymorphism in C Using Function Pointer Tables

In low-level systems programming, it is common for multiple components to expose the same logical operations but require different implementations. C does not support object-oriented polymorphism, but similar behavior can be achieved using **function pointer tables**.

A structure can store pointers to functions representing operations that different modules or drivers implement independently. At runtime, the system assigns the appropriate implementation to the structure and invokes the operation through the function pointer.

```
struct ops {
    int (*init)(void);
    int (*read)(char *buf);
    int (*write)(const char *buf);
};
```

Each hardware driver implements these operations separately.

Instead of writing conditional logic like:

```
if(device == UART)
    uart_read();

if(device == SPI)
    spi_read();
```

a driver can register its own implementation in the structure and the system simply calls:

```
driver->read(buffer);
```

This design pattern enables **clean driver abstraction and modular architecture**.

The Linux kernel uses this pattern extensively. One of the most well-known examples is:

```
struct file_operations
```

Each device driver populates this structure with pointers to its own implementations of:

- read
- write
- open
- release

The kernel then invokes these functions through the dispatch table rather than checking device types.

Challenge

Design a driver interface that allows multiple hardware drivers to register their implementations of initialization and data operations.

Requirements:

• Define a structure that stores function pointers for init, read, and write.

• Implement two drivers:

- UART driver
- SPI driver

• Each driver must provide its own implementation of the operations.

• Register the driver implementations in the function pointer table during initialization.

• Invoke driver operations through the dispatch table using a pointer to the structure.

• Ensure that the correct implementation is executed at runtime depending on the registered driver.

• Do not use conditional statements (if or switch) when invoking driver operations.

Recovering a Parent Structure from an Embedded Node

In high-performance systems programming, data structures are often designed to avoid unnecessary memory allocations. One common technique is the use of **intrusive data structures**, where the node used by the data structure is embedded directly inside the object rather than allocated separately.

For example:

```
struct device {
    int id;
    struct list_head list;
};
```

In this structure, **list_head** represents the linked list node embedded inside the device structure.

When traversing a linked list implemented in this way, the traversal logic typically operates on the list node itself. As a result, the program may only receive a pointer to the embedded node:

```
struct list_head *node;
```

However, the program often needs access to the **containing structure** that owns the node.

To obtain the address of the parent structure, pointer arithmetic can be used. Since the location of the embedded member within the structure is known, the base address of the structure can be recovered by subtracting the offset of the member.

Something like:

```
(type *)((char *)ptr - offsetof(type, member))
```

This calculation moves the pointer backward from the embedded node to the start of the containing structure.

This technique allows generic data structures to operate independently of the objects that contain them.

The Linux kernel relies heavily on intrusive linked lists. Instead of storing external nodes, structures embed a list_head member and participate directly in the linked list.

To recover the containing structure during traversal, the kernel uses the well-known macro:

```
container_of()
```

This macro allows the kernel to recover the address of the containing structure from a pointer to one of its members.

Challenge

Design a small program that demonstrates how to recover the address of a structure from a pointer to one of its embedded members.

Define the following structure:

```
struct device {
    int id;
    struct list_head list;
};
```

Assume that during a linked list traversal you only have a pointer to the list node:

```
struct list_head *node;
```

Compute the offset of the list member inside the device structure.

• Use pointer arithmetic to recover the address of the containing struct device.

• Verify the result by printing the id field from the recovered structure.

• Ensure the solution works without storing any additional reference to the parent structure.

Returning Newly Created Objects Using Double Pointers

Functions in C receive arguments **by value**, meaning a function cannot directly modify the caller's pointer unless it receives the **address of that pointer**.

This is commonly required when a function allocates a new object and must return the pointer to the caller.

Example structure representing a process:

```
struct task {
    int pid;
    struct task *parent;
};
```

If a function is responsible for creating a new process structure, it must:

- allocate memory
- initialize the structure
- return the pointer to the caller

To update the caller's pointer variable, the function must receive a **double pointer**.

Example signature:

```
int create_task(struct task **child);
```

Process creation in Linux occurs through the **fork()** mechanism.

Internally, the kernel eventually reaches the function:

```
_do_fork()
```

This function allocates and initializes a new process structure before linking it into the process hierarchy.

Understanding pointer ownership is essential when designing APIs that create kernel objects.

Challenge

Design a function:

```
create_task(struct task **child)
```

that:

- allocates memory for a new task structure
- assigns a process identifier
- links the new task to a parent process
- returns the pointer to the caller through the double pointer argument.

Ensure that the caller receives the newly created task pointer after the function returns.

Dynamic Memory Allocation with Double Pointers

When a function must allocate memory and return the pointer to the caller, it must receive the address of the caller's pointer. This is achieved using a **double pointer**.

A common example is the **getline()** function, which reads an input line and dynamically allocates or resizes the buffer storing the data.

Example usage:

```
char *buffer = NULL;
size_t size = 0;

getline(&buffer, &size, stdin);
```

The function prototype is:

```
ssize_t getline(char **lineptr, size_t *n, FILE *stream);
```

lineptr → pointer to the caller's pointer
*lineptr → actual buffer pointer

Because the function may allocate new memory using **malloc()** or expand the buffer using **realloc()**, the pointer itself may change.

Passing the **address of the pointer** allows the function to update it safely.

The GNU C Library implements **getline()** internally through the function **__getdelim()**.

Excerpt from the glibc implementation (stdio-common/getdelim.c):

```
ssize_t __getdelim(char **lineptr, size_t *n, int delimiter, FILE *fp)
{
    if (*lineptr == NULL || *n == 0)
    {
        *n = 120;
        *lineptr = malloc(*n);
        if (*lineptr == NULL)
            return -1;
    }
}
```

Here the function modifies *lineptr, which directly updates the caller's pointer.

lineptr → &buffer

*lineptr → buffer

Challenge

Design a program that demonstrates how a function can allocate memory and return the pointer to the caller using a double pointer.

Requirements:

• Define a function that receives a pointer to a pointer for a character buffer.

• The function should allocate memory dynamically using malloc().

• If the buffer already exists, the function should resize it using realloc().

• The function must update the caller's pointer to the newly allocated memory.

• In the main program, pass the address of the pointer to the function.

• After the function returns, verify that the caller can access the allocated memory.

• Print the contents of the buffer to confirm that the pointer update propagated correctly.

Multi-Level Pointer Indirection in Argument Vectors

Command-line arguments in C programs are passed as arrays of string pointers. Each argument is a pointer to a string, and the collection of arguments itself is represented as an array of pointers.

Example:

```
char *args[] = {
    "ls",
    "-l",
    "/home",
    NULL
};
```

When this structure is passed to system calls such as execve(), the argument vector becomes a multi-level pointer structure.

Representation:

args → array of char*
args[i] → pointer to string
args[i][j] → individual character

If the program stores the address of this pointer array:

```
char **argv = args;
char ***p = &argv;
```

multiple layers of pointer dereferencing appear. Understanding how pointer arithmetic interacts with these layers is critical when working with argument vectors and similar pointer tables.

During program execution, the Linux kernel processes argument vectors when loading a new process.

In the Linux source code, argument vectors are walked and copied during the execve() implementation. The kernel iterates through the pointer array and copies each argument string into the new process memory space.

Example snippet from the Linux kernel (fs/exec.c):

```
static int count(struct user_arg_ptr argv, int max)


{
    int i = 0;
    const char __user *p;

    for (;;) {
        if (get_user(p, argv.ptr + i))
            return -EFAULT;
        if (!p)
```

```
            break;
        if (i >= max)
            return -E2BIG;
        i++;
    }
    return i;
}
```

Here:

argv.ptr → pointer to array of pointers
argv.ptr[i] → pointer to argument string

This pointer chain is equivalent to navigating multiple levels of indirection.

Challenge

Design a small program that demonstrates how argument vectors can be traversed using multiple levels of pointer indirection.

Requirements:

• Define an argument vector similar to:

```
{"ls", "-l", "/home", NULL}
```

• Create the following pointer hierarchy:

```
char **argv
char ***p
```

• Use pointer dereferencing to access:

```
*p
**p
***p
```

• Advance the argument vector using pointer arithmetic on the appropriate pointer level.

• Demonstrate how advancing the pointer changes which argument string is referenced.

• Print the currently referenced argument after modifying the pointer chain.

Article Written By: Yashwanth Naidu Tikkisetty